Sprint & Task Breakdown Plan

Vehicle Parking System

Group 14 — Problem Statement No. 10

Project	Vehicle Parking System (MERN Stack)
Document	Implementation Sprint & Task Breakdown Plan
Version	1.0
Date	04-10-2026
Status	Draft — working document
References	Vehicle Parking System — Software Requirements Specification v1.0 Vehicle Parking System — Software Test Plan v1.0 Vehicle Parking System — Software Architecture and Design Specification v1.0
Team	Siddhesh Dattatray Koditkar — PES2UG24CS502 — Project Lead & Development Lead Sumukh M Gowda — PES2UG24CS537 — Backend & Integration Lead Sparsha Arun — PES2UG24CS513 — Frontend & Test Automation Lead Sinchana Kulkarni — PES2UG24CS504 — Security & Release Lead

Table of Contents

1. Introduction

2. Build-Order Philosophy

3. Dependency Overview Diagram

4. Sprint 1 — Foundation & Independent Core Logic (Oct 2–8)

5. Sprint 2 — Core Business Services (Oct 9–15)

6. Sprint 3 — API Wiring, Real-Time & Admin (Oct 16–22)

7. Sprint 4 — Hardening, Simulation Console & Load Test (Oct 23–29)

8. Master Task Dependency Table

9. Appendix — Testing Cheat-Sheet

1. Introduction

1.1 Purpose

This document turns the VPS SRS, STP, and SAD into an executable build order: concrete tasks, who owns each one, what it depends on, and what it unblocks. The STP's four-week calendar (Section 7) is preserved — this plan fills in what happens inside each of those weeks and why it's sequenced that way.

1.2 How to Read This Document

Each sprint is broken into tracks. A track is a unit of work one person can own start-to-finish. Tracks within the same sprint are safe to build in parallel unless a “Depends On” column points to another track in that same sprint — a handful of tracks do (Socket.io wiring, in particular, genuinely can't start until the controllers it taps into exist). Every task carries its SRS/SAD requirement IDs so the reason for the task is always one lookup away.

1.3 Team & Role Mapping

Task ownership below follows the leads already established in the STP: Siddhesh coordinates and floats to unblock whichever track is critical-path that week; Sumukh owns backend/allocation logic; Sparsha owns frontend and test automation; Sinchana owns auth, security, and the load test.

2. Build-Order Philosophy

2.1 Where to Start: Data Before Behaviour, Behaviour Before Endpoints

In a MERN project the dependency chain almost always runs the same direction: your API routes are thin wrappers around service functions, and those service functions only make sense once the shape of the data they read and write is fixed. Start anywhere else and you end up rewriting whatever you built first the moment the schema settles. So the build order is Models → Services → Controllers/Routes → Real-time wiring → Frontend real-data wiring → Hardening.

The one deliberate exception is the fare calculation logic: it's pure arithmetic with no database dependency at all (given an entry time, exit time, and vehicle type, it returns a number), so it can — and should — be written and unit-tested on day one, in parallel with the schema work, rather than waiting for a service layer to “arrive” at it.

2.2 Frontend Does Not Wait for Backend

The most common avoidable delay on a small team is treating frontend as something that starts after the API is done. It doesn't have to: the SAD's API Design section (4.3) is already a complete contract — every endpoint's request and response shape is written down. Sparsha can build every screen against that contract using mocked responses starting in Sprint 1, and swap each mock for a real fetch call the moment the matching endpoint lands in Sprint 3. The UI being visually complete and the UI being really wired to the server are two separate milestones, and only the second one has a backend dependency.

2.3 Testing in Practice: Jest, Supertest, and CI Are Three Different Jobs

These three tools are often conflated, so it's worth being precise, since the STP commits to all three:

- Jest is what writes and runs a test, locally, on demand. It's the test runner and assertion library — both unit and integration tests are Jest tests.
- Supertest is a library Jest tests use to make real HTTP requests against your actual Express app in-process — that's what turns a Jest test from a unit test into an integration test.
- GitHub Actions (a workflow) doesn't write or run tests itself — it automatically triggers npm test (your Jest suite, unit and integration both) on every push or pull request, and can block a merge if anything fails. It's automation around the tests, not a test framework.

Concretely:

Unit test — no server, no database

Tests one function in isolation. Dependencies are mocked or simply don't exist yet.

```
// server/services/fareService.test.js
const { calculateFare } = require('./fareService');

test('first hour is free for a 4-wheeler', () => {
  const entry = new Date('2026-10-02T10:00:00Z');
  const exit  = new Date('2026-10-02T10:45:00Z');
  expect(calculateFare(entry, exit, '4W')).toBe(0);
});
```

Integration test — real Express app, disposable MongoDB

Boots the actual app and a throwaway in-memory MongoDB instance (mongodb-memory-server — no shared test database to manage, no state leaking between runs) and fires a real HTTP request at a real route.

```
// server/routes/entry.integration.test.js
const request = require('supertest');
const { MongoMemoryServer } = require('mongodb-memory-server');
const mongoose = require('mongoose');
const app = require('../app');

let mongod;
beforeAll(async () => {
  mongod = await MongoMemoryServer.create();
  await mongoose.connect(mongod.getUri());
});
afterAll(async () => {
  await mongoose.disconnect();
  await mongod.stop();
});

test('POST /api/entry/vehicles allots a 4W spot', async () => {
  const res = await request(app)
    .post('/api/entry/vehicles')
    .set('Authorization', `Bearer ${entryGateToken}`)
    .send({ plateNumber: 'KA01AB1234', vehicleType: '4W', gateId: 'Entry-1' });

  expect(res.status).toBe(201);
  expect(res.body.ticketId).toBeDefined();
});
```

CI — runs both suites automatically on every push

```
# .github/workflows/test.yml
name: Test Suite
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '24'
      - run: npm ci
      - run: npm test
```

System tests (full entry-to-exit across simulated gates, per STP 5.1) stay a scripted manual multi-tab walkthrough for this project, consistent with the tooling already committed to in the STP — introducing a browser-automation framework like Playwright would be a reasonable future upgrade but isn't assumed here.

3. Dependency Overview Diagram

Each box is one track from the tables in Sections 4–7. An arrow means a hard dependency; two boxes with no arrow between them are safe to build at the same time, even across different people.

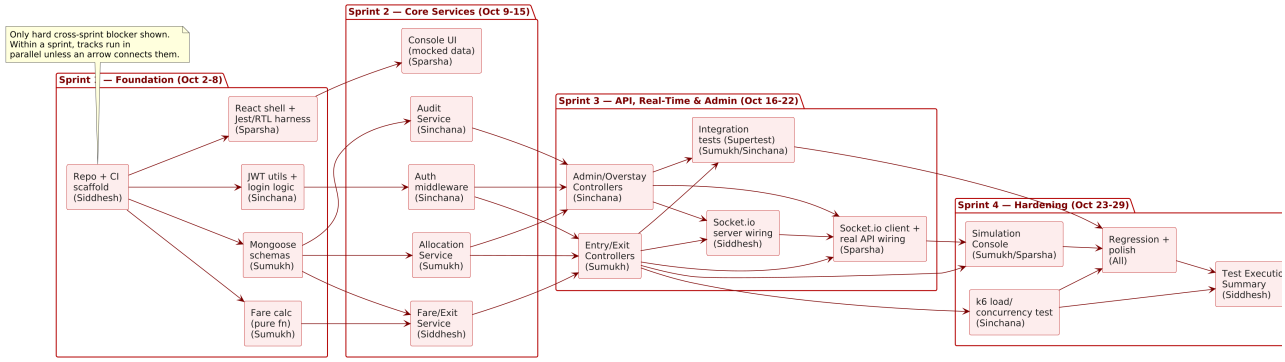


Figure 1 — Task dependency graph across all four sprints

4. Sprint 1 — Foundation & Independent Core Logic

Goal: every piece of ground-truth state (schemas) and every piece of logic with no external dependency (fare math, JWT utilities) exists and is unit-tested. Nothing in this sprint depends on anything else in this sprint except the repo existing.

Track	Owner	Task	Depends On	Unlocks	Suggested Tests
T1	Siddhesh	Repo scaffold: client/server folders, ESLint/Prettier, .env.example, branch strategy, empty CI workflow file.	—	Everything below	—
T2	Sumukh	Mongoose schemas: Vehicle/Spot, Transaction, User, AuditLog, FareTierConfig (per SAD 3.6).	T1	All Sprint 2 services	Schema validation unit tests
T3	Sumukh	Fare calculation as a pure function: tiered rate, first-hour-free, exact duration, no rounding (VPS-F-033, VPS-F-034).	T1	S2 Fare/Exit Service	Unit: boundary values at each tier, 2W vs 4W
T4	Sinchana	JWT sign/verify utilities, bcrypt password hashing (PRJ-SR-002, PRJ-SR-003).	T1	T5, S2 Auth middleware	Unit: valid/expired/tampered token
T5	Sinchana	Login logic: validate credentials against User model, issue JWT, lockout after 5 failed attempts (VPS-F-001..005).	T2, T4	S2 Auth middleware	Unit: lockout threshold, gate-scope claim present
T6	Sparsha	React app scaffold, routing shell for all four consoles, Jest + React Testing Library harness wired up.	T1	S2 Console UI	Smoke test: each route renders

Definition of Done: schemas finalized and reviewed by the whole team (they're the contract everyone else builds against); fare calculation and JWT utilities pass their unit tests in CI; React shell runs with route stubs.

5. Sprint 2 — Core Business Services

Goal: every service the API layer will eventually call exists and is independently unit-tested. Allocation and Fare/Exit are deliberately owned by different people — they touch different models and don't call each other, so there's no reason to serialize them.

Track	Owner	Task	Depends On	Unlocks	Suggested Tests
T1	Sumukh	Allocation Service: zone-matched query,	S1-T2	S3 Entry/Exit	Unit: zone matching,

Track	Owner	Task	Depends On	Unlocks	Suggested Tests
		atomic findOneAndUpdate, FCFS by server-received timestamp, lot-full handling (VPS-F-010..014).		Controllers	lot-full case. Concurrency case deferred to Sprint 4's k6 test.
T2	Siddhesh	Fare/Exit Service: ticket-ID lookup, plate-number fallback lookup, lost-ticket penalty, wire Sprint 1's fare function to real entry/exit timestamps (VPS-F-030..037).	S1-T2, S1-T3	S3 Entry/Exit Controllers	Unit: lookup-miss case, lost-ticket penalty applied
T3	Sinchana	Audit Service: append-only log writer, one entry per state-changing action (VPS-F-060..062).	S1-T2	S3 Admin/Overstay Controllers	Unit: one entry written per call, no edit/delete method exposed
T4	Sinchana	Auth middleware: protect routes, enforce per-gate credential scoping server-side (PRJ-SR-004, PRJ-SR-005).	S1-T5	S3 Entry/Exit + Admin Controllers	Unit: request to wrong gate rejected even with a valid token
T5	Sparsha	Build Entry/Exit/Admin console UI components against mocked API responses (matching SAD 4.3 exactly); component tests.	S1-T6	S3 real-data wiring	RTL: renders correctly given a mocked response shape

Definition of Done: all four services pass unit tests in isolation (no Express app required to run them); auth middleware rejects cross-gate requests in a unit test, not just "works when I tried it once"; console UI is visually complete end to end, still running on mock data.

6. Sprint 3 — API Wiring, Real-Time & Admin

Goal: the Sprint 2 services become real HTTP endpoints, Socket.io broadcasts state changes live, and the frontend swaps mocks for the real thing. This sprint has the most genuine intra-sprint sequencing — Socket.io wiring (T3) and the frontend's real-data swap (T4) can't start until the controllers they depend on exist, so they realistically begin a day or two into the sprint rather than on day one.

Track	Owner	Task	Depends On	Unlocks	Suggested Tests
T1	Sumukh	Wire Entry/Exit Controllers to Express routes: POST /api/entry/vehicles, GET /api/exit/lookup/:ticketId, GET /api/exit/search, POST /api/exit/process (SAD 4.3.2–4.3.3).	S2-T1, S2-T2, S2-T4	T3, T4, T6	Integration (Supertest): full entry→allocate, full exit→fare flows
T2	Sinchana	Wire Admin/Overstay Controllers: force-release, overstay notify, analytics reads, audit-log read, config writes (SAD 4.3.4–4.3.6).	S2-T1, S2-T3, S2-T4	T3, T4, T6	Integration: operator token rejected on admin routes (403)
T3	Siddhesh	Socket.io server: emit spot:update, overstay:alert, lot:snapshot on every relevant mutation in T1/T2 (SAD 4.3.8).	T1, T2 (first endpoints live)	T4	Integration: emitted event payload matches the REST response that triggered it
T4	Sparsha	Socket.io client wiring + swap mocked API calls for real ones; build Admin analytics charts (Recharts) and audit-log viewer.	T3 (socket), T1/T2 (REST)	Sprint 4 polish	RTL: live state update re-renders the lot map
T5	Siddhesh	Integration test suite for everything wired this sprint, using mongodb-memory-server (STP 6.3).	T1, T2	Sprint 4 regression baseline	—

Definition of Done: a full entry-to-exit cycle works through the real UI, with two entry gates and two exit gates open as separate logged-in sessions,

and every session's lot map updates within 2 seconds of a change made in any other session (VPS-F-021, VPS-NF-001).

7. Sprint 4 — Hardening, Simulation Console & Load Test

Goal: prove the concurrency claim under real load, finish the dev-only Simulation Console, and close out the STP's exit criteria.

Track	Owner	Task	Depends On	Unlocks	Suggested Tests
T1	Sinchana	k6 script: fire N simultaneous POST /api/entry/vehicles across both entry gates, assert zero duplicate spot assignments (VPS-NF-002).	S3-T1	Exit criteria	This is the test — k6 run itself is the deliverable
T2	Sumukh	Simulation Console backend: synthetic-arrival generator, runtime traffic-rate and fare-tier overrides (VPS-DEV-001..003).	S3-T1	T3	Unit: generator respects configured rate
T3	Sparsha	Simulation Console UI; final red/white theme + accessibility pass (colour-plus-label spot indicators) across all consoles.	T2, S3-T4	Demo readiness	RTL smoke test on the console
T4	All (Siddhesh coordinates)	Regression pass (re-run full Jest suite), bug triage and fixes, demo dry-run across 5 simultaneous tabs.	Everything above	Submission	Full suite green in CI
T5	Siddhesh	Finalize the Test Execution Summary / Log (STP Section 8) from this sprint's CI runs and the k6 result.	T1, T4, S3-T5	Submission	—

Definition of Done: matches the STP's Section 5.4 exit criteria exactly — all High-priority test cases pass, zero critical defects open, RTM shows nothing High-priority unresolved.

8. Master Task Dependency Table

Every task above, flattened into one reference table. Task IDs are scoped to their sprint (e.g. S2-T2 = Sprint 2, Track 2).

Task ID	Sprint	Owner	Task	Depends On	SRS / SAD Ref
S1-T1	1	Siddhesh	Repo + CI scaffold	—	—
S1-T2	1	Sumukh	Mongoose schemas	S1-T1	SAD 3.6
S1-T3	1	Sumukh	Fare calc (pure fn)	S1-T1	VPS-F-033/034
S1-T4	1	Sinchana	JWT + bcrypt utils	S1-T1	PRJ-SR-002/003
S1-T5	1	Sinchana	Login logic	S1-T2, S1-T4	VPS-F-001..005
S1-T6	1	Sparsha	React shell + Jest/RTL harness	S1-T1	—
S2-T1	2	Sumukh	Allocation Service	S1-T2	VPS-F-010..014
S2-T2	2	Siddhesh	Fare/Exit Service	S1-T2, S1-T3	VPS-F-030..037
S2-T3	2	Sinchana	Audit Service	S1-T2	VPS-F-060..062
S2-T4	2	Sinchana	Auth middleware	S1-T5	PRJ-SR-004/005
S2-T5	2	Sparsha	Console UI (mocked)	S1-T6	SAD 4.3
S3-T1	3	Sumukh	Entry/Exit Controllers	S2-T1, S2-T2, S2-T4	SAD 4.3.2/4.3.3
S3-T2	3	Sinchana	Admin/Overstay Controllers	S2-T1, S2-T3, S2-T4	SAD 4.3.4-4.3.6
S3-T3	3	Siddhesh	Socket.io server wiring	S3-T1, S3-T2	SAD 4.3.8
S3-T4	3	Sparsha	Socket.io client + real-data wiring	S3-T3	VPS-F-021/022
S3-T5	3	Sumukh/ Sinchana	Integration test suite	S3-T1, S3-T2	STP 6.3

Task ID	Sprint	Owner	Task	Depends On	SRS / SAD Ref
S4-T1	4	Sinchana	k6 load/concurrency test	S3-T1	VPS-NF-002
S4-T2	4	Sumukh	Simulation Console backend	S3-T1	VPS-DEV-001..003
S4-T3	4	Sparsha	Simulation Console UI + polish	S4-T2, S3-T4	—
S4-T4	4	All	Regression + bug fixes	All above	STP 5.4
S4-T5	4	Siddhesh	Test Execution Summary	S4-T1, S4-T4	STP 8

9. Appendix — Testing Cheat-Sheet

Quick reference, consolidated from Section 2.3:

Question	Answer
Where does a unit test run?	In Jest, in your head basically — no server, no database, mocked inputs where needed.
Where does an integration test run?	In Jest too, but using Supertest to call your real Express app, against a disposable in-memory MongoDB (mongodb-memory-server).
What does GitHub Actions do?	Runs npm test (both suites) automatically on every push/PR — it doesn't write or define tests itself.
What proves the concurrency requirement?	Neither Jest suite alone — that's what the k6 load test in Sprint 4 (S4-T1) is specifically for.
What covers a full multi-tab demo flow?	A manual System-level walkthrough per STP 5.1 — not automated in this plan.